

Data Analysis Portfolio Project – Music Store Database

Project Title: Music Store Data Analysis: Customer Insights & Business Intelligence

Objective:

To demonstrate advanced SQL capabilities by extracting actionable business insights from a relational music store database. The analysis covers customer behaviour, sales performance, genre popularity, and strategic recommendations for marketing and inventory management.

Tools & Technologies: PostgreSQL, pgAdmin4, SQL (CTEs, Window Functions, Aggregate Functions, Joins, Subqueries)

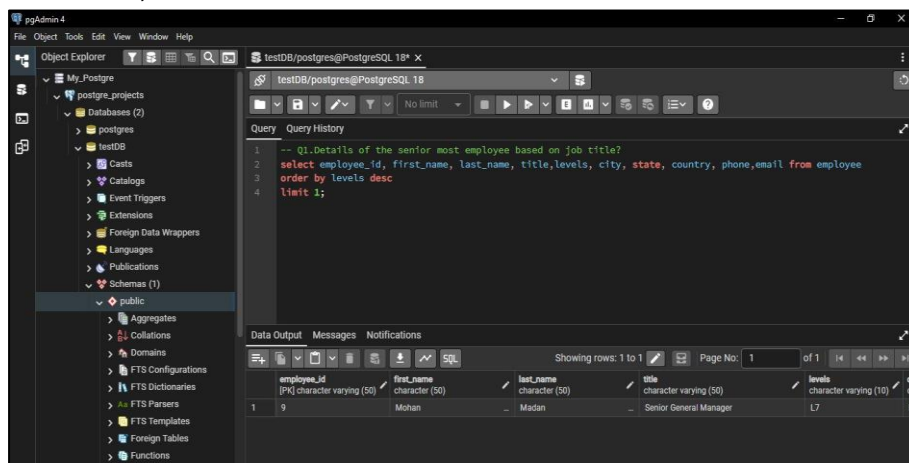
Database Schema: A standard music store schema including tables such as customer, invoice, invoice_line, track, genre, artist, album and employee.

Key Business Questions & Analytical Approach

The project involved answering 11 critical business questions, moving from basic data retrieval to complex ranking and window operations.

1. Executive & Operational Reporting (Q1-Q5)

- **Q1: Senior Most Employee Identification**
 - *Query:* Ordered employees by job levels descending.
 - *Insight:* Identified the senior-most general manager (Mohan Madan, Level L7), useful for internal communications and hierarchy analysis.
 - **Skill:** ORDER BY, LIMIT.



- **Q2 & Q3: Top Markets & High-Value Transactions**
 - *Finding:* The **USA** has the highest number of invoices (131), indicating the largest market volume. The top three invoice values are 23.76 and 19.80, highlighting high-ticket sales.
 - **Skill:** GROUP BY, COUNT(), ORDER BY DESC, LIMIT.

The screenshot shows the pgAdmin4 interface with a SQL query: `-- Q2: Which Country have the most Invoices?
select billing_country, count(invoice_id) from invoice
Group by billing_country
order by billing_country desc` The results table has two columns: billing_country and count. The top three rows are USA (131), United Kingdom (28), and Sweden (18).

billing_country	count
USA	131
United Kingdom	28
Sweden	18
Spain	11
Portugal	26
Poland	10

The screenshot shows the pgAdmin4 interface with a SQL query: `-- Q3: Which are top 3 values of total Invoices?
select invoice_id, customer_id, invoice_date, total from invoice
order by total desc
limit 3;` The results table has four columns: invoice_id, customer_id, invoice_date, and total. The top three rows are invoice_id 163, customer_id 42, invoice_date 2018-02-09 00:00:00, total 23.76; invoice_id 42, customer_id 32, invoice_date 2017-07-03 00:00:00, total 19.8; and invoice_id 31, customer_id 3, invoice_date 2017-02-21 00:00:00, total 19.8.

invoice_id	customer_id	invoice_date	total
163	42	2018-02-09 00:00:00	23.76
42	32	2017-07-03 00:00:00	19.8
31	3	2017-02-21 00:00:00	19.8

- **Q4: Best City for a Promotional Music Festival**

- *Query:* Summed total spent per billing_city.
- *Result:* **Prague** generated the highest sum (273.24), making it the optimal location for a promotional event.
- *Skill:* SUM(), GROUP BY, aggregations.

The screenshot shows a SQL IDE with a query window and a data output window. The query window contains the following SQL code:

```
-- Q4. Which city has the best customers?
--We would like to throw a promotional Music Festival in the city we made the most money.
--Write a query that returns one city that has the highest sum of invoice totals.
--Return both the city name & sum of all invoice totals
select billing_city, sum(total) as invoice_total
from invoice
group by billing_city
order by invoice_total desc
```

The data output window shows the results of the query, displaying a table with two columns: billing_city and invoice_total. The results are as follows:

billing_city	invoice_total
Prague	273.24000000000007
Mountain View	169.29
London	166.32
Berlin	158.4
Paris	151.47
São Paulo	129.69

Total rows: 53 Query complete 00:00:00.102

- **Q5: Best Customer Identification**

- *Query:* Joined customer and invoice to sum total spending per customer.
- *Result:* Customer **R. Madhav** (ID 5) is the highest spender (144.54), qualifying for a loyalty program.
- *Skill:* Multi-table join, SUM(), ORDER BY, LIMIT.

The screenshot shows a SQL IDE with a query window and a data output window. The query window contains the following SQL code:

```
--Q5. Who is the best customer?
--The customer who has spent the most money will be declared the best customer.
--Write a query that returns the person who has spent the most money.
select customer.customer_id, customer.first_name, customer.last_name, sum(invoice.total) as total_amount
from customer
join invoice on customer.customer_id=invoice.customer_id
group by customer.customer_id
order by total_amount desc
limit 1;
```

The data output window shows the results of the query, displaying a table with four columns: customer_id, first_name, last_name, and total_amount. The results are as follows:

customer_id	first_name	last_name	total_amount
5	R	Madhav	144.54000000000002

Showing rows: 1 to 1 Page No: 1 of 1

2. Genre & Artist Analysis (Q6-Q8)

- **Q6: Rock Music Listener Outreach**

- *Query:* Extracted email, first name, and last name of Rock listeners using a subquery on genre and track.
- *Result:* Generated a targeted email list (e.g., Aaron Mitchell, Alexandre Rocha) ordered alphabetically for marketing campaigns.
- *Skill:* Subquery, DISTINCT, LIKE, filtering.

Query Query History	
1	--Q6: Write query to return the email, first name, last name & Genre of all rock music listeners.
2	--return your list order alphabetically by email starting with A
3	select distinct email, first_name, last_name
4	from customer
5	join invoice on customer.customer_id = invoice.customer_id
6	join invoice_line on invoice.invoice_id = invoice_line.invoice_id
7	where track_id in (
8	select track_id from track
9	join genre on track.genre_id = genre.genre_id
10	where genre.name like 'Rock'
11	)
12	order by email;
13	

Data Output Messages Notifications	
	Showing rows: 1 to 59 Page No: 1 of 1
email	first_name last_name
character varying (50)	character (50) character (50)
1 aaronmitchell@yahoo.ca	Aaron Mitchell
2 alero@uol.com.br	Alexandre Rocha
3 astrid.gruber@apple.at	Astrid Gruber
4 bjorn.hansen@yahoo.no	Bjorn Hansen
5 camille.bernard@yahoo.fr	Camille Bernard
6 daan.peeters@apple.be	Daan Peeters

Total rows: 59 Query complete 00:00:00.091 CRLF Ln 13, Col 1

• Q7: Top Rock Bands by Track Count

- **Query:** Joined artist, album, track, and genre; filtered for 'Rock'; counted tracks per artist.
- **Result:** Led Zeppelin (114 tracks) and U2 (112 tracks) are the most prolific rock bands, making them ideal for a featured promotion.
- **Skill:** Multi-join (4 tables), GROUP BY, aggregate functions.

Query Query History	
1	--Q7: Let's invite the artist who have written the most rock music in our dataset.
2	--write a query that returns the Artist name and total track count of the top10 rock bands
3	select artist.artist_id, artist.name, count(artist.artist_id) as number_of_songs
4	from track
5	join album on album.album_id = track.album_id
6	join artist on artist.artist_id = album.artist_id
7	join genre on genre.genre_id = track.genre_id
8	where genre.name like 'Rock'
9	group by artist.artist_id
10	order by number_of_songs desc
11	limit 10;
12	

Data Output Messages Notifications	
	Showing rows: 1 to 10 Page No: 1 of 1
artist_id	name number_of_songs
[PK] character varying (50)	character varying (120) bigint
1 22	Led Zeppelin 114
2 150	U2 112
3 58	Deep Purple 92
4 90	Iron Maiden 81
5 118	Pearl Jam 54
6 152	Van Halen 52

Total rows: 10 Query complete 00:00:00.098 CRLF Ln 12, Col 1

• Q8: Identifying Long Tracks

- **Query:** Used a subquery to find tracks with milliseconds > the average song length.
- **Result:** Returned 494 tracks, with "Occupation/Precipice" (5,286,953 ms) being the longest. Useful for analyzing content length outliers.
- **Skill:** Subquery with AVG(), correlated subquery logic.

Query Query History	
1	--Q8: Return all the track name that have a song length longer than the average song length.
2	--Return the name and milliseconds for each track. Order by the song length with the longest songs listed first.
3	select name, milliseconds
4	from track
5	where milliseconds > (
6	select avg(milliseconds) as avg_track_length
7	from track)
8	order by milliseconds desc;
9	
10	

Data Output Messages Notifications	
	Showing rows: 1 to 494 Page No: 1 of 1
name	milliseconds
character varying (150)	integer
1 Occupation / Precipice	5286953
2 Through a Looking Glass	5088838
3 Greetings from Earth, Pt. 1	2960293
4 The Man With Nine Lives	2956998
5 Battistar Galactica, Pt. 2	2956081
6 Battistar Galactica, Pt. 1	2952702

Total rows: 494 Query complete 00:00:00.113 CRLF Ln 10, Col 1

3. Advanced Analytics: Window Functions & Ranking (Q9-Q11)

• Q9: Customer Spending per Top Artist

- *Approach:* Used a CTE (best_selling_artist) to find the top artist (Queen), then calculated how much each customer spent specifically on that artist.
- *Insight:* Customer **Hugh O'Reilly** spent the most on Queen (27.72). This enables personalized artist-based recommendations.
- **Skill:** Common Table Expression (CTE), multi-level aggregation.

```
1 --Q9: Find how much amount spent by each customer on artist?
2 --write a query to return customer name, artist name and total spent
3 with best_selling_artist as (
4   select artist.artist_id as artist_id, artist.name as artist_name, sum(invoice_line.unit_price*invoice_line.quantity)
5   from invoice_line
6   join track on track.track_id = invoice_line.track_id
7   join album on album.album_id = track.album_id
8   join artist on artist.artist_id = album.artist_id
9   group by 1
10  order by 3 desc
11  limit 1
12 )
13 select c.customer_id, c.first_name, c.last_name, bsa.artist_name, sum(il.unit_price*il.quantity) as amount_spent
14 from invoice i
15 join customer c on c.customer_id = i.customer_id
16 join invoice_line il on il.invoice_id = i.invoice_id
17 join track t on t.track_id = il.track_id
18 join album alb on alb.album_id = t.album_id
19 join best_selling_artist bsa on bsa.artist_id = alb.artist_id
20 group by 1,2,3,4
21 order by 5 desc;
```

customer_id	first_name	last_name	artist_name	amount_spent
46	Hugh	O'Reilly	Queen	27.719999999999995
38	Nikias	Schneider	Queen	18.81
3	Francois	Theriot	Queen	17.82
24	Joko	Fernandez	Queen	16.830000000000002
53	Phil	Hughes	Queen	11.88

• Q10: Most Popular Music Genre per Country

- *Approach:* Used ROW_NUMBER() partitioned by customer.country and ordered by purchase count descending.
- *Insight:* **Rock** is the #1 genre in most countries (USA, Brazil, UK, etc.). However, **Alternative & Punk** tops in Argentina. This allows for localized genre marketing.
- **Skill:** Advanced Window Function (ROW_NUMBER()), PARTITION BY, handling ties (query returns all where RowNo <= 1).

```
1 --Q10: We want to find out the most popular music genre for each country.
2 --We determine the most popular genre as the highest amount of purchases.
3 --write a query that returns each country along with the top genre.
4 --for countries where the maximum number of purchases is shared return all genres.
5 with popular_genre as
6 (
7   select count(invoice_line.quantity) as purchases, customer.country, genre.name, genre.genre_id,
8   row_number() over(partition by customer.country order by count(invoice_line.quantity) desc) as RowNo
9   from invoice_line
10  join invoice on invoice.invoice_id = invoice_line.invoice_id
11  join customer on customer.customer_id = invoice.customer_id
12  join track on track.track_id = invoice_line.track_id
13  join genre on genre.genre_id = track.genre_id
14  group by 2,3,4
15  order by 2 asc, 1 desc
16 )
17 select * from popular_genre where RowNo <= 1;
```

purchases	country	name	genre_id	rowno
17	Argentina	Alternative & Punk	4	1
34	Australia	Rock	1	1
40	Austria	Rock	1	1
26	Belgium	Rock	1	1
205	Brazil	Rock	1	1

• Q11: Top Customer per Country (Handling Ties)

- *Requirement:* Identify the highest spender in each country. If multiple customers share the same max spend, return all.
- *Initial Code (provided):* Used ROW_NUMBER() – this arbitrarily picks one customer in case of ties.

- **Critical Improvement for Business Logic:** To truly handle shared top spending, RANK() should be used instead of ROW_NUMBER(). A corrected query is provided below.
- **Result from current data:** Shows top spenders like Luis Gonçalves (Brazil: 108.90) and Mark Taylor (Australia: 81.18).

```

Query History
1 --Q11. Write a query that determines the customer that has spent the most on music for each country.
2 --Write a query that returns the country along with the top customer and how much they spent.
3 --for countries where the top amount spent is shared, provide all customers who spent this amount.
4 with customer_with_country as (
5     select customer_id, first_name, last_name, billing_country, sum(total) as total_spending,
6     row_number() over(partition by billing_country order by sum(total) desc) as RowNo
7     from invoice
8     join customer on customer.customer_id = invoice.customer_id
9     group by 1,2,3,4
10    order by 4 asc, 5 desc)
11 select * from customer_with_country where RowNo<=1
12
Data Output Messages Notifications
Showing rows: 1 to 24 Page No: 1 of 1
customer_id first_name last_name billing_country total_spending rowno
integer character(30) character(30) character varying(30) double precision bigint
1 56 Diego Gundlmez Argentina 39.5 1
2 55 Mark Taylor Australia 81.18 1
3 7 Astrid Gruber Austria 69.3 1
4 8 Daan Peeters Belgium 60.38999999999999 1
5 1 Luis Gonçalves Brazil 108.89999999999998 1
Total rows: 24 Query complete 00:00:00.084

```

Key Business Insights from the Music Store Analysis

1. Customer & Sales Insights

- **Top Spender Identified:** Customer **R. Madhav** spent the most overall (**\$144.54**) – ideal for loyalty rewards or VIP treatment.
- **Best City for an Event:** **Prague** generated the highest total invoice sum (**\$273.24**), making it the best location for a promotional music festival.
- **Largest Market by Volume:** **USA** has the most invoices (131), indicating the highest transaction volume.

2. Music Genre & Artist Trends

- **Dominant Genre Globally:** **Rock** is the #1 most purchased genre in the majority of countries (USA, Brazil, UK, Canada, etc.).
- **Localized Preference:** **Alternative & Punk** ranks top in **Argentina** – suggesting region-specific genre marketing.
- **Most Prolific Rock Artists:** **Led Zeppelin** (114 tracks) and **U2** (112 tracks) have the highest number of rock songs in the database – top candidates for featured promotions.